Advanced Object Oriented Programming

Xingjie ZHANG

아주대학교
AJOU UNIVERSITY

아주대학교
AJOU UNIVERSITY

Class - 2

Instructor: Xingjie ZHANG

Email : xzhang@ajou.ac.kr

Wechat : runningSnaile

CONTENTS

01 Constructor – Multiple Definitions

02 Shallow Copy and Deep Copy

03 Copy Constructor

04 Static Members

아주대학교 AJOU UNIVERSITY

01

Constructor Multiple Definitions

Constructor – Multiple Definitions

- Like regular methods, constructors can define multiple constructors with **different parameter** configurations.
  - Only one constructor is invoked when the new operation is performed.
- **Call other constructors from within a constructor.**

Constructor – Multiple Definitions

- _constructorOver

```
class UserData {
    private String name;
    private int age;

    UserData() {
        System.out.println("UserData()");
    }

    UserData(String name, int age) {
        this();
        System.out.println("UserData(String, int)");
        this.name = name;
        this.age = age;
    }
}
```

```
UserData()
UserData(String, int)
```

Constructor – Multiple Definitions

- Practice
  - MyString Class

```java
class MyString {  2 usages
    private byte[] string;  8 usages

    public String getString() {  2 usages
        if(string == null || string.length <= 0) {
            return "(null)";
        }

        return new String(string);
    }
    public void setString(String param) { string = param.getBytes(); }

    public int length() {  2 usages
        if(string == null)
            return 0;
        return string.length;
    }

    public int compareTo(String param) {  1 usage
        if(string == null)
            return -1;

        return param.compareTo(new String(string));
    }
}
```

```java
public class Main {
    public static void main(String[] args) {
        MyString str = new MyString();
        System.out.println(str.length());
        System.out.println(str.getString());

        str.setString("Hello Java");
        System.out.println(str.getString());
        System.out.println(str.length());
        System.out.println(str.compareTo("Hello Java"));
    }
}
```

```
0
(null)
Hello Java
10
0

Process finished with exit code 0
```

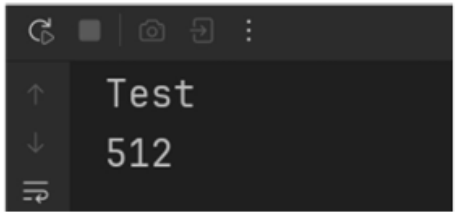
Constructor – Multiple Definitions

- 添加如下三个构造函数到 MyString Class
  - MyString()
  - MyString(String)
  - MyString(int)

```
public class Main {
    public static void main(String[] args) {
        MyString str1 = new MyString("Test");
        System.out.println(str1.getString());
        MyString str2 = new MyString(512);
        System.out.println(str2.getString());
    }
}
```

How to convert Int to String?

- **String.format()**
- **Integer.toString()**

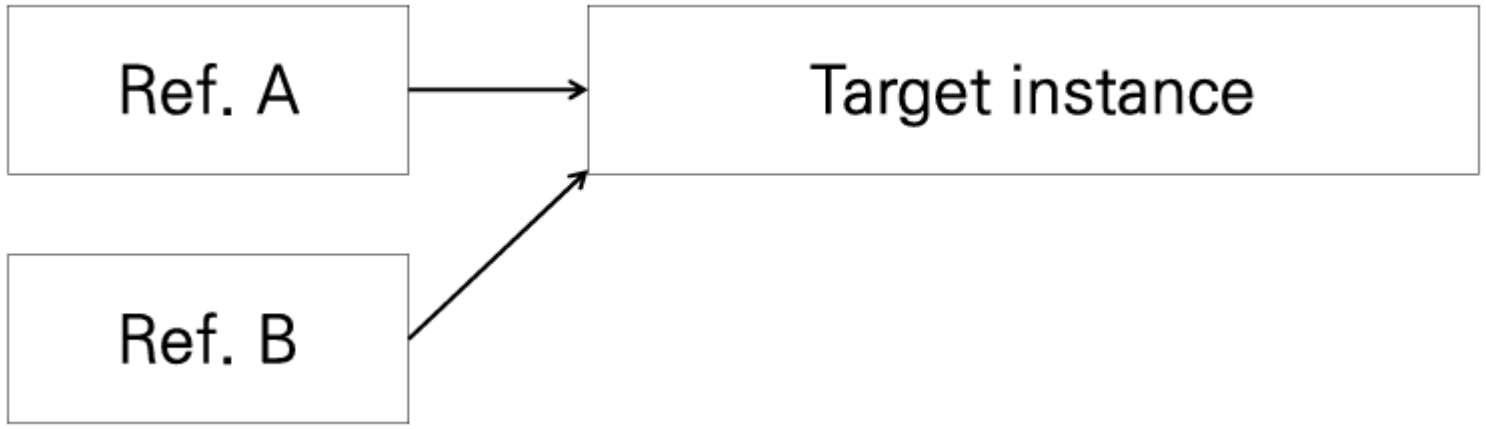


02

Shallow Copy and Deep Copy

Shallow Copy

- The object of the reference is not copied and increased, but only the reference is increased.
- the cause of the side effect.



Shallow Copy

- _shallowCopy

```
class Address {
    Address(String address, String phone) {
        this.address = address;
        this.phone = phone;
    }

    public String address;
    public String phone;
}
```

Shallow Copy

```
class User {
    private String name;
    private Address address;

    User(String name, String address, String phone) {
        this.name = name;
        this.address = new Address(address, phone);
    }

    public Address getAddress() {
        return address;
    }

    public String getName() {
        return name;
    }

    public void copy(User rhs) {
        this.name = rhs.name;
        this.address = rhs.address;
    }
}
```

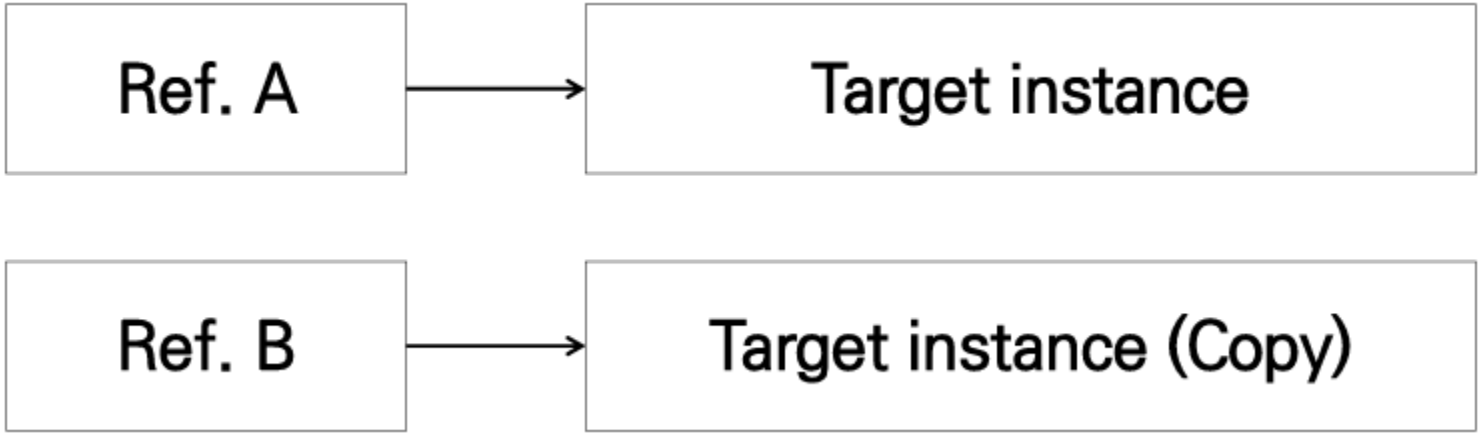
Shallow Copy

```
public class Main {
    public static void main(String[] args) {
        User user1 = new User("Hosung", "Hanam", "010-1111-1111");
        User user2 = new User("Hoon", "Seoul", "010-2222-2222");

        System.out.println(user1.getName());
        System.out.println(user1.getAddress().address);
        System.out.println(user1.getAddress().phone);

        user1.copy(user2);
        user2.getAddress().phone = "010-3333-3333";
        user2.getAddress().address = "Busan";

        System.out.println(user1.getName());
        System.out.println(user1.getAddress().address);
        System.out.println(user1.getAddress().phone);
    }
}
```

```
Hosung
Hanam
010-1111-1111
Hoon
Busan
010-3333-3333
```

Deep Copy

- The original itself is also **newly allocated and copied**.
- Unlike shallow copy, the two originals and two references exist separately.
- No side-effect errors.
- Implement a **copy constructor** when dynamically allocating instances with references as members.



Deep Copy

- _deepCopy

```
public void copy(User rhs) {
    this.name = rhs.name;
    //this.address = rhs.address;
    this.address.address = rhs.address.address;
    this.address.phone = rhs.address.phone;
}
```

```
Hosung
Hanam
010-1111-1111
Hoon
Busan
010-3333-3333
```

```
Hosung
Hanam
010-1111-1111
Hoon
Seoul
010-2222-2222
```

03

Copy Constructor

Copy Constructor

- A constructor function suitable for creating copies of objects.
- There is a way to create a clone() method, but its specifications are ambiguous and exception handling is complex.

Copy Constructor

- _copyConstructor

```
class MyTest {
    private int[] array = null;
    public MyTest(int size) {
        array = new int[size];
    }

    public MyTest(MyTest rhs) {
        this(rhs.array.length);
        this.deepCopy(rhs);
    }
```

Copy Constructor

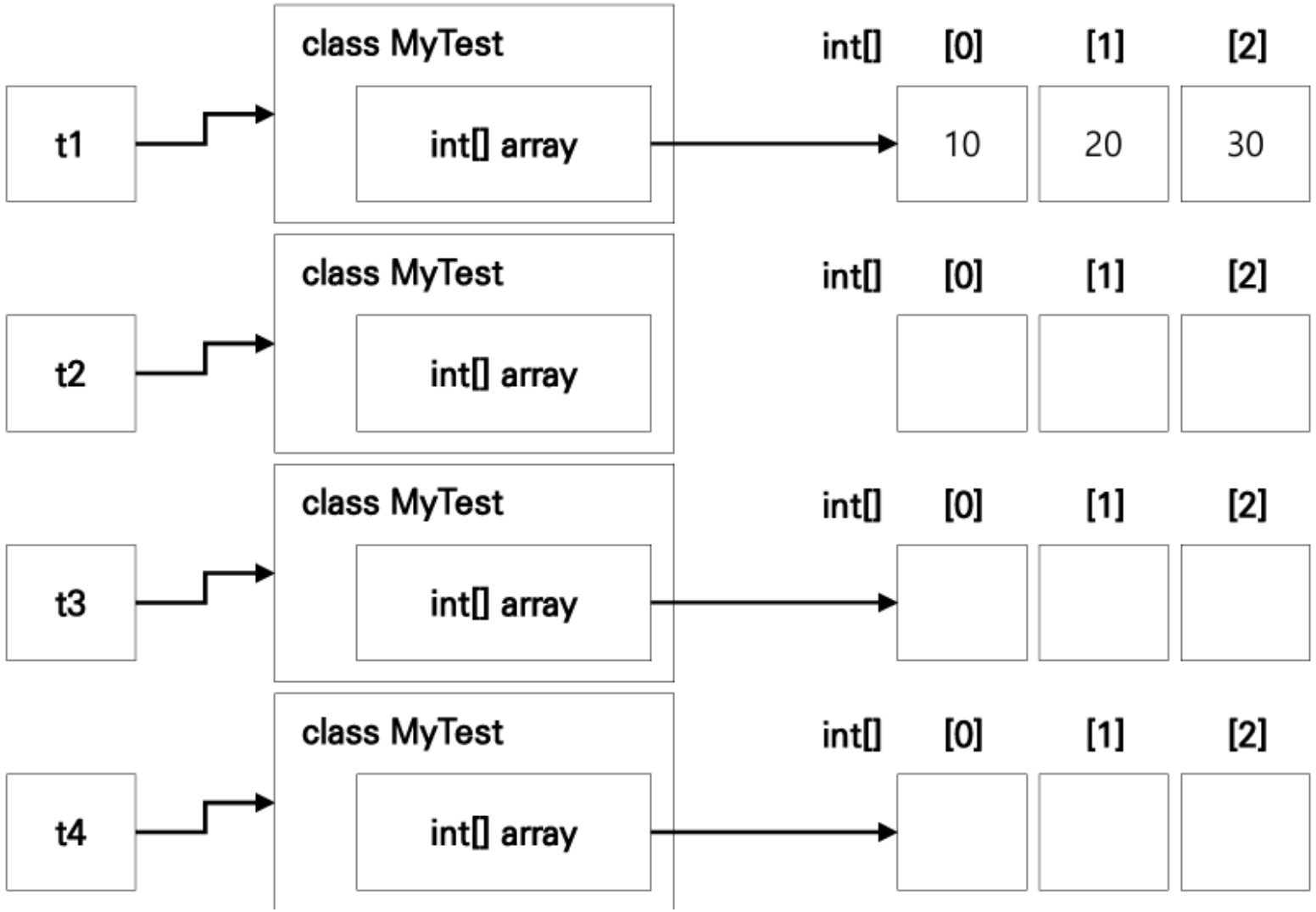
```
    public int getAt(int index) {
        return array[index];
    }

    public void setAt(int index, int value) {
        array[index] = value;
    }

    public void shallowCopy(MyTest rhs) {
        array = rhs.array;
    }

    public void deepCopy(MyTest rhs) {
        //array = src.clone();
        for(int i = 0; i < rhs.array.length; ++i)
            array[i] = rhs.array[i];
    }
}
```

Copy Constructor

```
public class Main {
    public static void main(String[] args) {
        MyTest t1 = new MyTest(3);
        t1.setAt(0, 10);
        t1.setAt(1, 20);
        t1.setAt(2, 30);

        MyTest t2 = new MyTest(3);
        t2.shallowCopy(t1);
        MyTest t3 = new MyTest(3);
        t3.deepCopy(t1);
        MyTest t4 = new MyTest(t1);

        t1.setAt(0, -1);
        System.out.println("t1[0]: " + t1.getAt(0));
        System.out.println("t2[0]: " + t2.getAt(0));
        System.out.println("t3[0]: " + t3.getAt(0));
        System.out.println("t4[0]: " + t4.getAt(0));
    }
}
```

```
t1[0]: -1
t2[0]: -1
t3[0]: 10
t4[0]: 10
```

Copy Constructor



Copy Constructor

- Pratice: 添加如下2个函数到 MyString Class
  - Void deepCopy(MyString rhs)
  - MyString(MyString rhs)

```
public class Main {
    public static void main(String[] args) {
        MyString src = new MyString("Hello");
        MyString dst = new MyString(src);

        src.setString("World");
        System.out.println(dst.getString());
    }
}
```

```
C:\Users\cx853\.jdks\
Hello
```

Temporary Object

- When a class becomes the return type of a function, a temporary object without a name is created.
- The String class creates a temporary object when performing addition operations.

Temporary Object

- _tmpObj

```
public class Main {
    public static void main(String[] args) {
        String left = "Hello";
        String right = "World";

        System.out.println((left + right).toUpperCase());
        String result = left + right;
        System.out.println(result);
    }
}
```

```
HELLOWORLD
HelloWorld
```

04

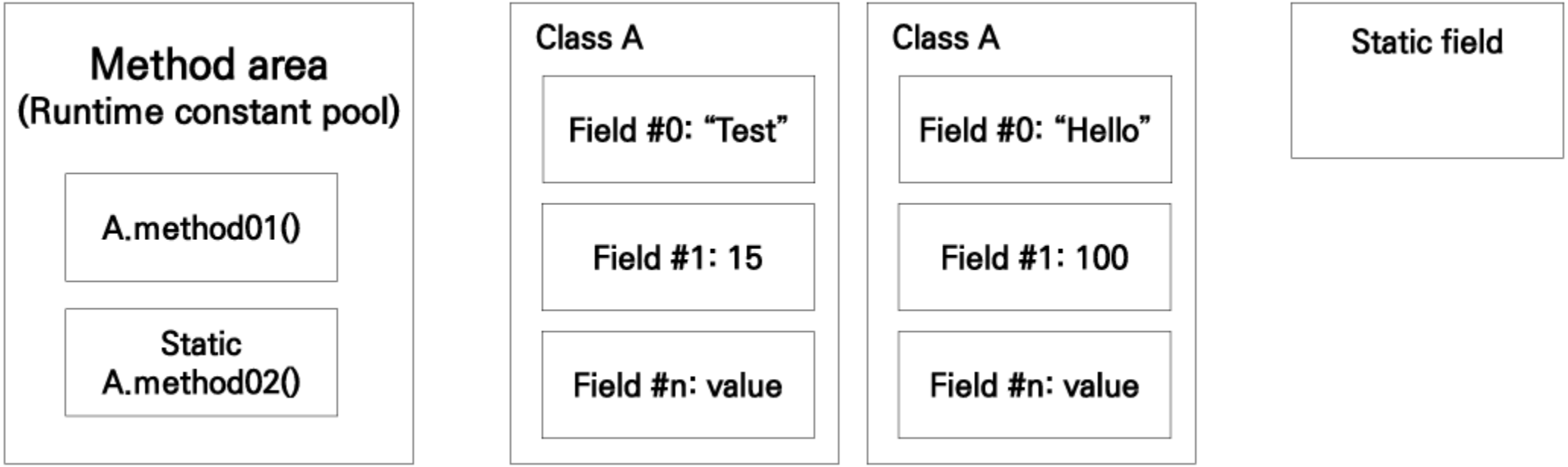
Static Members

Static Members

- **Static field members**: Can exist independently, even without a class instance.
- **Static method members**: Unlike regular methods, they can be called without an instance declaration.
- This reference cannot be used.
- Static field members are often used as symbolic constants by declaring them final.

Static Members

- Static fields are independent of instance memory, and method code does not vary from instance to instance; only one exists.



Static Members

- Field access issues in static methods

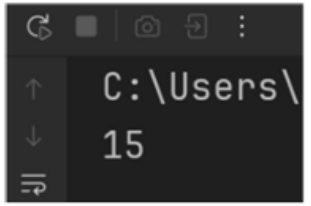
```
class MyTest {
    private int data;
    public void setData(int param) { this.data = param; }
    public int getData() { return this.data; }

    public static void printData() {
        System.out.println(data);
    }
}
```

```
java: non-static variable data cannot be referenced from a static context
```

Static Members

- Field access issues in static methods

```
class MyTest {
    private int data;
    public void setData(int param) { this.data = param; }
    public int getData() { return this.data; }

    public static void printData(MyTest obj) {
        System.out.println(obj.data);
    }
}

public class Main {
    public static void main(String[] args) {
        MyTest test = new MyTest();
        test.setData(15);
        test.printData(test);
    }
}
```



Static Members

- _staticField

```
class MyScore {
    private static int limit = 80;

    public void setLimit(int limit) {
        MyScore.limit = limit;
    }

    public int getLimit() {
        return limit;
    }
}

public class Main {
    public static void main(String[] args) {
        MyScore s1 = new MyScore();
        MyScore s2 = new MyScore();
        s1.setLimit(90);
        System.out.println(s2.getLimit());
    }
}
```

```
C:\Users
90
```

Copy Constructor

- Practice
  - 为了确认MyString Class的Instance总量，把构造函数设置为private权限，并添加如下静态成员：
  - MyString newString(int)
  - MyString newString(String)
  - Static int count

```
public class Main {
    public static void main(String[] args) {
        //MyString src = new MyString("Hello");
        MyString s1 = MyString.newString(10);
        MyString s2 = MyString.newString("Test");

        System.out.println(s1.getString());
        System.out.println(s2.getString());
        System.out.println(MyString.getCount());
    }
}
```

```
10
Test
2
```

Quiz

- 1. What is the primary role of a class constructor?
  A. It defines the class structure.
  B. It initializes the object's state.
  C. It frees the object from memory.
  D. It creates methods that perform calculations.

Quiz

- 2. When performing a "shallow copy" of an object that contains references to other objects, what is copied?
  A. New instances of all contained objects.
  B. The reference values of contained objects.
  C. Only primitive data values.
  D. The entire object instance contents.

Quiz

- 3. Which of the following statements about static members in Java is correct?
  A. They belong uniquely to each object instance.
  B. They are accessed using the `this` keyword.
  C. All instances of a class share a single copy.
  D. They allow direct access to non-static instance variables.

Quiz

- 4. When are invisible temporary objects most commonly implicitly created in Java?
  A. When explicitly calling a constructor with the `new` keyword
  B. When declaring a static variable
  C. When a method returns a new object or an object is created as a result of an operation
  D. When assigning a literal value to a primitive type variable

Quiz

- 5. If all of a class's constructors are declared private, what is the typical way to obtain an instance of that class from external code?
  A. Using the `new` keyword directly
  B. Calling a public static factory method
  C. Inheriting the class
  D. Accessing a public static variable containing the instance

Q & A

THANK YOU